FULL-CYCLE WEB3 FANTASY RPG

ON-CHAIN ECONOMY AS GAMEPLAY.

Собственный Jetton MAT и NFT-аватары встроены в прогрессию, боевые характеристики и рынок — активы не украшают игру, а меняют её правила.

TELEGRAM WEBAPP | BROWSER / PWA | TON MAINNET



4 CUSTOM FUNC CONTRACTS	41 ISSUED NFT AVATARS	123/123 IPFS ASSETS AVAILABLE	0.278s BATTLE FIRST EVENT	1 983 MAT LIVE DEX SWAP RESULT

01–04 КЛЮЧЕВЫЕ ИНЖЕНЕРНЫЕ РЕШЕНИЯ

01 PLAYABLE NFT
Владение проверяется в TON; on-chain атрибуты автоматически пересчитывают игровые stats и снимаются после transfer.

02 GAME → CHAIN BRIDGE
Вывод MAT и перенос XP защищены Ed25519-подписью, TTL, уникальным tx_id, replay protection и safe mode.

03 SELF-OWNED ECONOMY
Цена связана с reserve / supply; реализованы mint, burn, buyback, 30% tax routing и минимальный резерв 10 TON.

04 EVENT-DRIVEN GAMEPLAY
WebSocket + Kafka + Redis hot-state при PostgreSQL source of truth; тяжёлый клиент вынесен за lazy startup boundary.

OW TRADE — LOWER ◂ MARKET ▸ UPPER
Market-keeper, а не спекулятивный бот: сравнивает коридор контракта с STON.fi / Omniston, действует только за границами буфера и пишет каждое решение в event store.

STACK TECHNOLOGY MAP

BLOCKCHAIN
TON · FunC · Jetton · NFT
Ed25519 · @ton/core · TonConnect
Toncenter · TonAPI · Wallet V5

BACKEND
Python 3.12 · FastAPI · DDD
Pydantic · Tortoise ORM · asyncio
REST · WebSocket · workers

FRONTEND & DATA
React 18 · Vite · Redux Toolkit
Telegram WebApp SDK · PWA
PostgreSQL · Redis · Kafka

MARKET & DELIVERY
STON.fi · Omniston · IPFS / Kubo
Docker Compose · Nginx · GitHub Actions
Sentry · Prometheus · Grafana

FROM INTERFACE TO INFRASTRUCTURE

I ENGINEERED THE WHOLE STACK.

Я связал клиент, доменную модель, события, данные и TON в один production-контур.

Не просто использовал технологии — спроектировал их роли, границы и взаимодействие.

ARCHITECTURE · BACKEND · FRONTEND · BLOCKCHAIN · DELIVERY

4	11	93	15	152	32
CUSTOM FUNC CONTRACTS	DEDICATED KAFKA WORKERS	APPLICATION EVENT HANDLERS	REDUX TOOLKIT SLICES	FASTAPI ROUTE DECLARATIONS	FASTADMIN MODEL PANELS

01–06 RUNTIME ARCHITECTURE I BUILT

01 / CLIENT — TELEGRAM + WEB
React 18 · Vite 5 · TypeScript · Redux Toolkit
React Router · Axios · React Context · i18next · MUI / Emotion · Framer Motion · PWA
STATE & DELIVERY
15 slices; Telegram WebApp и browser/PWA из общей кодовой базы.

02 / CORE — API + DOMAIN
Python 3.12 · FastAPI · Pydantic v2 · asyncio
DDD · Ports & Adapters · DI · Repository contracts · Uvicorn / Gunicorn · REST / WebSocket
BOUNDARIES
Domain / Application / Infrastructure / Interfaces без смешивания ответственности.

03 / EVENTS — KAFKA BACKBONE
Kafka · FastStream · aiokafka · Kafka UI
topic per event · consumer groups · keyed ordering · manual ACK · 11 isolsted worker processes
EVENT-DRIVEN
Бои, игроки, кланы, финансы, Materia, квесты и WebSocket развязаны событиями.

04 / HOT STATE — REDIS RUNTIME
redis.asyncio · cache · Pub/Sub · streams · dedupe
battle hot-state TTL · invalidation · status streaming · idempotency · delivery confirm / retry
FOUR ROLES
Redis работает не «просто кешем», а real-time транспортом и слоем надёжности.

05 / DATA + OPS — PERSISTENCE
PostgreSQL 17 · asyncpg · Tortoise ORM · PgBouncer
Aerich · repository adapters · transactions · FastAdmin · 32 models · 4 custom actions
SOURCE OF TRUTH
Постоянное игровое состояние, connection pooling и операционная админка.

06 / ON-CHAIN — TON + CONTENT
FunC · Jetton · NFT · TonConnect · TON Blueprint / Sandbox
@ton/core · @ton/ton · tonsdk · tonutils · Toncenter JSON-RPC · TonAPI · Ed25519 · IPFS / Storacha
CUSTOM PROTOCOL
1 717 LOC: Materia Minter/Wallet + Avatar Collection/Item и TypeScript wrappers.

CODE IMPLEMENTED ADAPTERS

NOT LOGOS — RESPONSIBILITIES

- `KafkaEventBroker` — topics, groups, manual ACK
- `RedisBattleStateCache` — TTL + precise invalidation
- `RedisEventStreaming` — async status channels
- `TonBlockchain` — RPC, BOC, tx parsing, signatures
- `KafkaPublisher` — event keys and ordering
- `RedisSocketPubSub` — delivery IDs + 3 retries
- `RedisDeduplicatorService` — 1-hour idempotency window
- `Web3NFTStorage` — Storacha / IPFS publishing

PRODUCTION DELIVERY

GitHub Actions → SSH / rsync → Docker Compose → Nginx

Stateful services поднимаются отдельно; API, background/blockchain и 11 Kafka workers пересоздаются как runtime-слой. Health checks входят в compose.

OBSERVABILITY & QUALITY

SENTRY · PROMETHEUS · GRAFANA · KAFKA LAG · PG EXPORTER · LOCUST · PYTEST